DS5006 Machine Learning — Team Project

Guidelines, Rules, and Evaluation

Fall 2026 · IIT Palakkad

September 22, 2026

In one paragraph. You will form a team of at most four, choose a dataset and a set of *scientific questions* about learning on that data, and then design and run experiments that answer them. You submit a report, reproducible code, and a log of any AI-tool use, and every member takes an oral exam on the whole project. The project is graded on the quality of the questions and the rigour of the evidence — not on how high your accuracy is.

1 Key dates

Milestone	What is due	Deadline
M0	Team registration (names, roll numbers, team name)	**[26th September, 12.00AM]**
M1	Project proposal (Appendix A template, max. 2 pages)	**[6th October, 12.00AM]**
M2	Mid-point check-in (progress note, 1 page, + preliminary results)	**[22nd October, 12.00AM]**
M3	Final submission: report, code, AI-use log	**[10th November, 12.00AM]**
M4	Oral exams (slots allotted per team)	**[10th and 12th November range]**

All submissions are through Moodle unless stated otherwise. A project whose questions change substantially after the initial project proposal must be justified.

2 Teams

- Teams are self-formed, with **at most four** members. Register your team by M0. Students not in a registered team by M0 will be assigned to teams by the instructor.
- Teams of fewer than four are permitted. The expected scope is the same per team, so smaller teams should scope accordingly.
- Team membership is frozen after M1. If a member drops the course, notify the instructor immediately; the project scope will be adjusted.
- If a member is not contributing, raise it with the instructor *early* — no later than M2. Concerns raised after the final submission are much harder to act on.

3 What an acceptable project looks like

A project is **not** "apply a machine learning model to a public dataset and report the accuracy." It is a small empirical study built around questions whose answers you do not know in advance.

Every project must satisfy all of the following.

1. **Scientific questions.** Two to three non-trivial questions (Section 4), each answered by a designed experiment.
2. **Machine learning.** The central object of study is a machine learning model (Section 5).
3. **Tabular data.** The data are tabular, or are images/text converted into a tabular representation (Section 6).
4. **Reproducibility.** Every number and figure in the report can be regenerated from the submitted code (Section 7).

4 Scientific questions

This is the most important part of the project, and the part most teams find hardest. Spend time on it.

What makes a question non-trivial

A good question passes all three tests:

1. **The answer is not predictable in advance.** If you could confidently guess the outcome before running anything, it is not worth a project.
2. **Answering it requires a designed manipulation.** You change one thing deliberately (add noise, remove features, shift the distribution, vary data size, perturb labels) while holding other things fixed, and observe the effect. Simply training several models and comparing them is not a manipulation.
3. **A "no" is as informative as a "yes."** If only one outcome would be interesting, the question is really a hope.

Warning signs

Your question is probably trivial if it:

- asks which model, architecture, optimiser, or hyperparameter setting is *best*;
- can be answered by reading a single table of test accuracies;
- replaces "model" with "algorithm" in a which-is-better question;
- has no stated condition under which the answer would be "no."

Examples

Standard benchmark datasets are permitted (Section 6). This makes one more kind of question trivial: **if the answer can be looked up in published results for that benchmark, it is not a project question.**

Example 1: feature importance in random forests

Trivial: Which features are most important for predicting the target, according to a random forest?
Non-trivial: Impurity-based feature importances in random forests are known to favour continuous and high-cardinality features (Strobl et al., *BMC Bioinformatics*, 2007). If we append synthetic features that are pure noise but differ in cardinality (binary, 10 levels, continuous), where do they rank under impurity-based importance and under permutation importance? Does adding them change the ranking of the real features, and does the effect depend on tree depth?
Why it works: noise features are, by construction, known to carry no information, so any importance they receive is measurable bias; the cardinality of the noise and the tree depth are controlled; "permutation importance also ranks them highly" would be a genuine surprise, and "the real ranking is unchanged" is a useful answer.

Example 2: images converted to embeddings

Trivial: Which pretrained encoder gives the highest accuracy with an SVM?
Non-trivial: CIFAR-100 labels each image with one of 100 fine classes and one of 20 superclasses. With images represented by frozen pretrained embeddings, and embeddings truncated by PCA to k dimensions, does a linear classifier lose fine-class accuracy faster than superclass accuracy? As k shrinks, do the errors become increasingly confusions *within* a superclass? Does k-NN on the same embeddings show the same pattern?
Why it works: the truncation level k is the manipulation; the label hierarchy lets you ask *what kind* of information is lost, not just how much accuracy; comparing a linear classifier with k-NN asks whether the answer depends on the classifier's geometry.

Example 3: label noise and k-NN

Trivial: What value of k gives the best k-NN accuracy?
Non-trivial: As a controlled fraction of training labels is flipped, how does the best k (chosen by cross-validation) shift? Is noise concentrated near the decision boundary — labels flipped on points a reference classifier is least confident about — more or less damaging than the same amount of uniformly random noise, and does increasing k compensate for both equally?
Why it works: the noise level and the noise location are both controlled while the data are held fixed; the uniform-versus-boundary comparison has no obvious answer in advance.

These examples are illustrations. **Do not submit them as your questions.**

Negative results are acceptable

A carefully designed experiment that shows a hypothesis is false earns full credit. Do not tune until something "wins." Grading rewards the design of the experiment and the honesty of the analysis, not the direction of the result.

5 Machine learning scope and baselines

This is a classical machine learning project. **No neural network may be trained, fine-tuned, or used as a predictor anywhere in the project.** The only exception is the use of frozen pretrained encoders to turn images or text into embeddings, under the rules below.

Permitted models

Any non-neural method, including but not limited to those covered in DS5006: linear and logistic regression and their regularised variants; k-nearest neighbours; naive Bayes; LDA and QDA; decision trees; random forests and other bagging methods; boosting (AdaBoost, gradient boosting, XGBoost, LightGBM, CatBoost); support vector machines and other kernel methods; Gaussian processes; clustering, mixture models, PCA and other dimensionality-reduction methods.

Not permitted

- Any multi-layer neural network, including scikit-learn's `MLPClassifier` and `MLPRegressor`.
- Deep tabular models and pretrained tabular predictors (e.g. TabNet, FT-Transformer, TabPFN).
- Fine-tuning, or adding trainable layers to, any pretrained model.
- Using a large language model or other generative model to produce predictions, labels, or features.

Embeddings of images and text

Pretrained encoders (e.g. CLIP, ResNet, sentence encoders) may be used *only* as frozen feature extractors.

- Extraction happens once, in a single script, `code/extract_embeddings.py`, which saves the embeddings as a table (`.npy`, `.parquet` or `.csv`). **All deep-learning library imports (`torch`, `tensorflow`, `transformers`, etc.) must be confined to this script.**
- That script performs inference only: no optimisers, no gradient steps, no added layers.
- Report the encoder, the exact checkpoint, which output is used (e.g. pooled output, final-layer `[CLS]` token), the embedding dimension, and the input preprocessing.
- Published precomputed embeddings may be used instead, with the source cited.
- The object of study is the classical model on the embeddings, not the encoder. Questions about which encoder is best are trivial (Section 4).

Reference models

Every experiment must include at least one simple reference model on identical splits: a majority-class (classification) or mean (regression) predictor and, where sensible, a regularised linear model. It is a yardstick for judging the size of your effects, not a competitor to beat.

Libraries

NumPy, SciPy, pandas, scikit-learn, statsmodels, XGBoost, LightGBM, CatBoost, matplotlib and seaborn may be used freely. PyTorch, Hugging Face libraries, `open_clip` and `sentence-transformers` may be used only inside `extract_embeddings.py`. You are not required to implement algorithms from scratch.

6 Data

- Data must be tabular, or images/text converted to a tabular representation. Describe the conversion in the proposal.

- You must be legally allowed to use the data. State the source and licence in the proposal.
- Any publicly available dataset can be used.

7 Compute and reproducibility

The submitted code will be executed on lab machines without a GPU.

Repetitions and uncertainty

- Every reported number is a mean over at least a few repetitions, reported with a measure of spread.
- **Repetitions must vary the data split** (repeated random splits or repeated k-fold cross-validation), not only the model's random seed. Many classical models — logistic regression, SVMs, k-NN — are deterministic given the data, so reseeding them alone produces zero spread and says nothing about reliability.
- When comparing conditions, use the same splits for every condition within a repetition, and report the paired difference with its spread.

Model selection

- Hyperparameters are chosen using training and validation data only (or nested cross-validation). The test portion of each split is used once, for the reported numbers.
- If a benchmark has an official test split, you may keep it as the fixed test set; repetitions then come from resampling or cross-validating the training portion. State which scheme you use.

Reproducibility

- Fix and report all random seeds.
- Provide a clear description of how to execute the python script.
- Provide a `requirements.txt` (or `environment.yml`) with pinned versions.
- Provide a `README.md` stating: how to obtain the data (for benchmarks, the exact source, version and loader call, e.g. an OpenML dataset ID), how to install, how to run each stage, expected run time per stage, and which script produces which table or figure.

8 Deliverables

M1: Proposal

Use the template in Appendix A. Max. 2 pages. The most common reason for revision is a trivial question.

M2: Mid-point check-in

One page: what has been done, preliminary results for at least one question, any change of plan, and each member's contribution so far. **[Ungraded but mandatory]**.

M3: Final submission

Upload a single zip file named `[TeamName]_DS5006_Project.zip` containing:

`report.pdf`	Final report, built from the provided LaTeX template
`report_src/`	LaTeX source of the report
`code/`	All code, with `README.md`, `requirements.txt`, entry script
`AI_USE_LOG.md`	AI-use log (Section 10)

Do not include large data files, trained model checkpoints, or virtual environments.

Page limit for the report: [8] pages excluding references, the contribution statement, the AI-use summary, and appendices.

9 Evaluation

What the report is graded on

1. Non-triviality and clarity of the scientific questions.
2. Whether each experiment actually isolates what the question asks about (controls, what is held fixed).
3. Correct handling of uncertainty (seeds, spread, not over-reading small differences).
4. Quality of the failure analysis: where and why the model fails.
5. Honesty about limitations: what the results do *not* show.

Raw performance numbers carry no marks on their own.

Oral exam

- Each team has a [20]-minute slot. **Every member must attend and answer.**
- **Any member may be asked about any part of the project**, including code they did not write and sections of the report they did not draft. Know the whole project.
- You may be asked to explain a specific line of code, modify a script, re-run an experiment, or predict the effect of a change.
- You may be asked about any interaction recorded in your AI-use log.
- The oral mark is individual. **A member who cannot explain the project may receive a lower overall project mark than the rest of the team**, **[up to and including zero for the project]**.

10 Use of AI tools

You may use AI tools (ChatGPT, Claude, Gemini, Copilot, etc.). **All use must be documented.**

Use	Status
Debugging, explaining errors, library usage	**[Permitted, logged]**
Generating code	**[Permitted, logged; you must be able to explain every line]**
Brainstorming or refining scientific questions	**[Permitted, logged]**
Drafting or rewriting report text	**[Permitted for language editing only]**
Generating results, figures, or analysis you did not run	**Not permitted**

- Maintain `AI_USE_LOG.md` throughout the project using the provided template. Log entries as you go; do not reconstruct them at the end.
- Each entry records: date, member, tool, purpose, the prompt (or a shared chat link), what you used from the output, and how you verified it.
- The report includes a short AI-use summary (template section provided).
- **Undisclosed AI use discovered during grading or the oral exam is treated as academic misconduct.** Disclosed use is never penalised in itself; what is graded is whether you understand and can defend the work.

11 Academic integrity and reuse

- Cite every external source: papers, blog posts, code repositories, tutorials.
- Reusing existing code is allowed if cited and if it is not the substance of your contribution. Your experiments and analysis must be your own.
- Reproducing a published paper is acceptable *only* as a starting point: your project must ask a new question of it.
- Collaboration between teams on ideas is fine; sharing code or results is not.

A Proposal template (max. 2 pages)

A1. Team. Team name; members with roll numbers.

A2. Title. A working title that states the question, not the method.

A3. Data. Source, size (rows × columns), target variable, and — for image/text data — how it will be converted to tabular form.

A4. Scientific questions. For each question (two or three):

(a) The question, in one sentence.

(b) Your hypothesis and why you hold it.

(c) The manipulation: what you will vary, over what range, and what you will hold fixed.

(d) The measurement: metric(s), number of seeds, how you will judge whether an effect is real.

(e) **What result would count as "no"?**

A5. Models. The algorithms/models that you will study.

A6. Risks. What could go wrong (e.g. the effect is too small to measure) and your fallback.

A7. Planned division of work. Who leads which part. (Every member remains responsible for understanding everything.)

B Final submission checklist

☐ Report uses the template and is within the page limit.

☐ Every question in the report matches the proposal.

☐ Every table and figure is produced by a script in `code/`, and the README says which.

☐ Results report mean and spread over the required number of seeds.

☐ The baseline appears on identical splits.

☐ Failure analysis and limitations sections are present.

☐ Contribution statement is filled in and agreed by all members.

☐ `AI_USE_LOG.md` is complete; the report includes the AI-use summary.

☐ `run_all` runs from a fresh environment.

☐ Zip file is named correctly and contains no data dumps or environments.